

Blatt 12

Software Qualität

Nichtfunktionale Tests

Luis Staudt

Aufgabe 1: Anzahl an gleichzeitigen Benutzern ermitteln

Ziel des Testszenarios

Für die neue Webshop-Applikation soll die **maximale Anzahl gleichzeitiger Benutzer** ermittelt werden, die das System noch zuverlässig (innerhalb der vereinbarten Service-Level) bedienen kann. Dazu werden vier Testarten kombiniert, die aufeinander aufbauen: Performanztest, Lasttest, Stresstest und Skalierbarkeitstest. Alle Tests laufen gegen eine produktionsnahe Umgebung; typische Nutzerabläufe (Anmelden, Suchen, Warenkorb, Bestellung/Bezahlung) werden als Lastprofil nachgebildet.

Einsatz der einzelnen Testarten

Performanztest (Baseline)

Einsatz: Zuerst wird das System unter geringer, definierter Last (z. B. wenige Benutzer) gemessen, um eine **Referenz (Baseline)** ohne Lasteinfluss zu gewinnen. So ist später erkennbar, wie sich die Werte bei steigender Last verschlechtern.

Metriken: Antwortzeit pro Transaktion (Mittel und p95/p99), Durchsatz (Requests/s), Ressourcenverbrauch im Ruhezustand.

Interpretation: liefert die Soll-Werte, gegen die alle weiteren Messungen verglichen werden.

Lasttest

Einsatz: Die Benutzerzahl wird schrittweise bis zur **erwarteten realistischen Last** erhöht und über längere Zeit konstant gehalten. Geprüft wird, ob das System die geplante Last innerhalb der Service-Level bewältigt.

Metriken: Antwortzeit (p95/p99) je Laststufe, Durchsatz, Fehlerrate (z. B. HTTP 5xx, Timeouts), Ressourcenauslastung (CPU, RAM, DB-Verbindungen).

Interpretation: Solange Antwortzeiten und Fehlerrate unter den Grenzwerten bleiben, gilt die jeweilige Benutzerzahl als zuverlässig bedienbar.

Stresstest

Einsatz: Die Last wird **über die erwartete Last hinaus** bis zum Überlastpunkt gesteigert, um die Belastungsgrenze und das Verhalten im Grenzbereich zu finden. Zusätzlich wird das Verhalten beim Zurücknehmen der Last (Recovery) beobachtet.

Metriken: Benutzerzahl am “Knickpunkt” (sprunghaft steigende Antwortzeit / Fehlerrate), Sättigungspunkt der Ressourcen, Erholungszeit nach der Spitze.

Interpretation: Der Punkt, ab dem die Service-Level verletzt werden, markiert die **Kapazitätsobergrenze**.

Skalierbarkeitstest

Einsatz: Es wird gemessen, wie die Kapazität mit **zusätzlichen Ressourcen** (mehr CPU/RAM bzw. zusätzliche Instanzen) wächst – also ob durch Hoch- bzw. Hinzuskalieren mehr gleichzeitige Benutzer möglich werden.

Metriken: maximal mögliche Benutzerzahl je Ausbaustufe, Durchsatz pro Ressourceneinheit, Effizienz der Skalierung (linear vs. abflachend).

Interpretation: zeigt, ob die ermittelte Obergrenze durch Skalierung erweiterbar ist und wo Engpässe (z. B. Datenbank) die Skalierung begrenzen.

Bestimmung der maximalen Benutzerkapazität

Die maximale Zahl gleichzeitiger Benutzer ergibt sich aus der höchsten Laststufe, bei der **alle definierten Akzeptanzkriterien gleichzeitig** eingehalten werden, z. B.:

- Antwortzeit p95 unter dem vereinbarten Grenzwert (z. B. $< 2\text{ s}$),
- Fehlerrate unter Schwelle (z. B. $< 1\%$),
- Ressourcenauslastung im stabilen Bereich (keine Sättigung).

Lasttest liefert den bestätigten Normalbetrieb, der Stresstest die harte Obergrenze; die Differenz dient als **Sicherheitsreserve**. Der Skalierbarkeitstest zeigt schließlich, ob und wie diese Grenze bei steigendem Bedarf wirtschaftlich angehoben werden kann.

Aufgabe 2: Zuverlässigkeit medizinischer Software

Zuverlässigkeit beschreibt die Fähigkeit der Software, ihre Funktionen unter festgelegten Bedingungen über einen bestimmten Zeitraum korrekt zu erbringen. Betrachtet werden die Aspekte **Reife**, **Fehlertoleranz** und **Wiederherstellbarkeit** am Beispiel der Software eines medizinischen Gerätes (z. B. Infusionspumpe, Beatmungs- oder Überwachungsgerät).

Teil 1: Die drei Aspekte

Reife (Maturity)

Allgemein: Grad, in dem die Software unter normalem Betrieb fehlerfrei arbeitet – also wie selten Ausfälle auftreten (gemessen z. B. über MTBF, Fehlerdichte, Ausfallrate). Reife entsteht durch ausgereiften, gut getesteten und im Feld bewährten Code.

Konkret beim Medizingerät: Eine hohe Reife bedeutet, dass die Pumpe im Dauerbetrieb zuverlässig die korrekte Dosis abgibt, ohne sporadische Abstürze oder Fehlfunktionen. Geringe Reife äußert sich in zufälligen Aussetzern während der Behandlung.

Fehlertoleranz (Fault Tolerance)

Allgemein: Fähigkeit, trotz auftretender Fehler (Software-Defekte, fehlerhafte Eingaben, Hardware-/Sensorausfälle) weiter ein definiertes, sicheres Verhalten zu zeigen, statt unkontrolliert auszufallen. Mittel sind z. B. Redundanz, Plausibilitätsprüfungen und ein sicherer Zustand (Fail-Safe).

Konkret beim Medizingerät: Fällt ein Sensor aus oder wird ein unplausibler Wert erkannt, schaltet das Gerät kontrolliert in einen sicheren Zustand und alarmiert das Personal, anstatt eine falsche Dosis abzugeben. Die Behandlung wird so nie unbemerkt fehlerhaft fortgeführt.

Wiederherstellbarkeit (Recoverability)

Allgemein: Fähigkeit, nach einem Ausfall den Betrieb und die betroffenen Daten/Zustände wiederherzustellen – gemessen z. B. über die Wiederherstellzeit (MTTR) und den Datenverlust.

Konkret beim Medizingerät: Nach einem Strom- oder Softwareausfall fährt das Gerät schnell wieder hoch, stellt die zuletzt gültigen Behandlungsparameter wieder her und nimmt die Therapie kontrolliert wieder auf, ohne dass eingestellte Werte verloren gehen.

Teil 2: Wichtigster Aspekt für ein Medizinprodukt

Am wichtigsten ist die **Fehlertoleranz**.

Begründung: Bei einem Medizinprodukt steht die **Patientensicherheit** an oberster Stelle. Reife reduziert die Häufigkeit von Fehlern, kann sie aber nie vollständig ausschließen; ebenso können Sensor- oder Stromausfälle nicht verhindert werden. Entscheidend ist daher, dass ein *auftretender* Fehler nicht zu einer gefährlichen Fehlfunktion führt. Eine fehlertolerante Steuerung verhindert genau das, indem sie unplausible Zustände erkennt, in einen sicheren Zustand wechselt und alarmiert – so wird eine Über-/Unterdosierung oder eine falsche Messung (medizinische Genauigkeit) mit unmittelbarer Gefahr für den Patienten vermieden. Fehlende Fehlertoleranz hat hier potenziell tödliche Konsequenzen, während ein seltener, aber sicher abgefangener Fehler beherrschbar bleibt.

Teil 3: Situationsabhängige Priorität

Welcher Aspekt dominiert, hängt von Einsatzart und Kritikalität ab:

- **Lebenserhaltende Geräte im Dauerbetrieb** (Beatmung): Reife wird sehr wichtig – jeder einzelne Aussetzer ist kritisch, das Gerät muss durchgehend fehlerfrei laufen.
- **Mobile/akkubetriebene oder störungsanfällige Umgebung** (Rettungswagen, Notfall): Wiederherstellbarkeit gewinnt an Bedeutung, da Strom- oder Verbindungsausfälle wahrscheinlicher sind und ein schnelles, datenerhaltendes Wiederanlaufen zählt.
- **Nicht-kritische Geräte** (z. B. reine Dokumentations-/Messsoftware ohne direkte Therapiewirkung): Hier dürfen kurze Ausfälle toleriert werden, Reife/Verfügbarkeit treten in den Vordergrund, da kein unmittelbares Patientenrisiko besteht.

In sicherheitskritischen Situationen bleibt die Fehlertoleranz jedoch die Grundvoraussetzung, auf der die übrigen Aspekte aufsetzen.

Aufgabe 3: Performanztest versus Ressourcennutzungstest

Beide gehören zu den **nichtfunktionalen Tests** und betrachten das Effizienzverhalten, verfolgen aber unterschiedliche Ziele: Der Performanztest fragt nach dem *Zeitverhalten* (wie schnell?), der Ressourcennutzungstest nach dem *Ressourcenverbrauch* (wie sparsam?).

Gegenüberstellung der Testziele

	Performanztest	Ressourcennutzungstest
Ziel	Zeit- und Reaktionsverhalten des Systems unter definierter Last bewerten	ermitteln, wie viele Ressourcen die Software für ihre Aufgaben verbraucht
Kernfrage	Wie schnell/leistungsfähig ist das System?	Wie effizient/sparsam nutzt es die Ressourcen?
Fokus	Antwortzeit, Durchsatz, Latenz	CPU, Speicher, Datenträger, Netz, Energie

Typische Metriken

Performanztest:

- Antwortzeit/Latenz (Mittel, p95/p99),
- Durchsatz (Transaktionen bzw. Requests pro Sekunde),
- Time-to-First-Byte / Time-to-Interactive.

Ressourcennutzungstest:

- CPU- und Speicherauslastung (inkl. Erkennung von Memory-Leaks),
- Datenträger- und Netz-I/O,
- Energie-/Akkuverbrauch, Anzahl belegter Verbindungen/Threads.

Nutzung der Ergebnisse zur Qualitätsverbesserung

Performanztest: Ergebnisse decken zu langsame Funktionen und Engpässe auf und führen zu Optimierungen (Caching, Indizes, Query-Tuning, Parallelisierung) sowie zur Festlegung/-Überwachung von Service-Levels.

Ressourcennutzungstest: Ergebnisse zeigen Verschwendung und Lecks und führen zu sparsamerer Implementierung, korrekter Ressourcenfreigabe und passender Dimensionierung der Hardware (Kostensenkung, Skalierbarkeit).

Beide ergänzen sich: Ein System kann schnell sein, dabei aber so viele Ressourcen verbrauchen, dass es teuer oder schlecht skalierbar ist – erst gemeinsam ergeben sie ein vollständiges Effizienzbild.

Konkrete Beispiele

- **Performanztest besonders relevant:** Webshop/Online-Portal, bei dem kurze Antwortzeiten unter Last über Nutzerzufriedenheit und Umsatz entscheiden; Echtzeit-/Handelssysteme mit harten Latenzvorgaben.
- **Ressourcennutzungstest besonders relevant:** Mobile App (Akku-/Speicherverbrauch) und Embedded-/IoT-Geräte mit knapper Hardware; Cloud-Dienste, bei denen der Ressourcenverbrauch direkt die Betriebskosten bestimmt; Langläufer-Dienste, bei denen Memory-Leaks über Tage auffallen.